International Journal of Information Management Data Insights 2 (2022) 100071

Contents lists available at ScienceDirect

International Journal of Information Management Data Insights

journal homepage: www.elsevier.com/locate/jjimei

Predicting reactions to anomalies in stock movements using a feed-forward deep learning network

Talal Al-Sulaiman

Saudi Real Estate Refinance Company, Prince Sultan University, Saudi Arabia

A B S T R A C T

Stock markets may be exposed to large movements for various reasons, which lead to negative or positive returns. This research aims to find if we can predict future prices after a shock? The problem is not concerned with the time series of stock prices but rather with stock price anomalies. Thus, a feed-forward deep learning network is more suitable for this problem than a recurrent neural network. We deploy deep learning mechanisms to predict stock prices at a given time following a significant movement given a few inputs. An experimental case study is implemented on stocks in the S&P 500 for particular variations of large movements. The results show that a feed-forward DNN can predict price anomaly reactions with a reasonable error margin. However, the mean absolute error is reduced when we aim to predict shorter periods. In addition, the prediction accuracy increases as the threshold are tightened.

1. Introduction

The academic community accepted the random walk assumption of stock movements as the foundation of financial theory. This assumption was followed by the development of the efficient market hypothesis (Fama, 1991). Even with this assumption of financial theory, many research studies among practitioners and academics have been conducted to predict the stock market's future prices. For example, the requirements of the efficient market hypothesis were loosened by the adaptive market hypothesis proposed by Lo [24] in 2004. Despite these efforts, financial asset prediction remains a challenging problem because stock movements are dynamic and influenced by many factors governed by the irrational human behaviors that introduce chaos into the market.

For decades, fundamental and technical analyses have been widely utilized to estimate stock prices. However, there are many financial ratios (Ehrhardt & Brigham, 2011) and technical indicators (Achelis, 2001). These ratios and indicators may conflict with each other and provide opposing signals. In other situations, these indicators and ratios occasionally provide identical signals, which create a source of redundancy in the analyses (Alsulaiman, 2013).

According to Kar & Dwivedi (2020), data availability and access to various platforms, is changing the nature of Information Systems. A large variety of potential data from different sources creates difficulty in explaining and detecting the complex relationships between inputs and outputs. Massive data, contradictions, redundancies, and the high likelihood of nonlinear relations between inputs and outputs make it unlikely that oversimplified models can provide robust conclusions. Modern technological developments and the evolution of statistical learning, which includes data mining, machine learning, and artificial intelligence techniques, have meant that researchers tend to develop algorithms that can provide the ability to discover patterns in stock movements. These trends have created a scientific branch of financial engineering called "algorithmic trading."

However, many predictive algorithms are subject to overfitting and provide average performance when different samples or time frames are considered. In other words, a predictive algorithm may not always be applicable to all cases. Deep learning is one aspect of machine learning that has proven to be a powerful tool in prediction and classification in diverse fields, such as computer vision, speech recognition, natural language processing, and text mining. Deep learning has been used to solve various financial problems, such as security pricing, portfolio construction, and risk management (Heaton, Polson, & Witte, 2016). In addition to solving many financial problems, it can have several advantages, such as the following (Heaton et al., 2016):

1. The input data are flexible for all relevant information.
2. It captures nonlinearity and complex interactions between inputs and outputs.
3. It can prevent overfitting.

The research is justified by the needs of financial institutes and investors to make critical decisions in the time of stock fluctuations. In this paper, we aim to measure the ability to predict future prices after an event. The key hypothesis of this research is that with few inputs, we may predict the future price on a given day under the condition that the stock is exposed to a large move. The main contribution of this research is to study the reaction to large stock movements over time using deep learning techniques. This notion is borrowed from physics, where Newton's third law of motion states that there is an equal and opposite reaction for every action. However, this may not be entirely true in

E-mail address: talsulaiman@srco.com.sa

https://doi.org/10.1016/j.jjimei.2022.100071
Received 6 August 2021; Received in revised form 24 March 2022; Accepted 25 March 2022
2667-0968/© 2022 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)

finance, as there may be no reaction to a movement force; movement may lead to a reaction in the opposite direction or generate momentum for further movement in the same direction. An additional contribution is to ensure the technique's robustness by empirically examining the techniques across various time scales and stocks in the S&P 500. We set a threshold for determining large stock movements. Three thresholds were specified in the experiments, at 2%, 4%, and 8%. Moreover, we show that using the ReLU activation function provides better results than using linear, sigmoid, softmax and tanh functions in our context. The comparisons between the activation functions are presented in Section 4.2.

Although different types of neural networks, such as recurrent neural networks (RNN) and long-short term memory networks, are widely used in predicting financial time series, they may not be suitable for this problem, as our focus is not on the stock time series but instead on the reactions to the stock anomaly. Thus, we use a deep feed-forward neural learning network for structure development. This paper develops various structures of deep learning neural networks to predict future prices on predetermined days.

The aim being is to predict the stock prices these days. These days, in the experiments, time scales are set to one day, two days, ten days, one month, and two months. The deep learning neural network consists of several hidden layers, each with various numbers of nodes. In this paper, we developed three forms of a feed-forward neural network. The first layer structure consists of three hidden layers with twenty nodes in the first layer, ten nodes in the second layer, and 5 five nodes in the third layer. The second form of the neural network consists of four hidden layers with forty nodes in the first layer and nodes of a similar number distribution of nodes into the first structure. The third structure has 80, 40, 20, 10, and five nodes in its first, second, third, fourth, and fifth layers, respectively layer, 40 nodes in the second layer, 20 nodes in the third layer, ten nodes in the fourth layer, and five nodes in the last layer.

The paper has the following structure: in Section 2, we introduce a literature review of machine learning and deep learning in finance. Section 3 describes the computational approach of the developed model, data collection. Sections 4 provide insight on the findings and results where it test for model robustness through a set of experiments in Section 4.1 and compare the proposed model to other machine learning algorithms in Section 4.2. Section 5 discuss the implications for theory and practice in Section 5.2 and the limitations and direction researches in Section 5.3. Section 6 provides the conclusion.

2. Literature review

There are a large number of financial services that serve billions of people around the globe. These services include commercial banking, investment banking, and insurance. Each of these services involves various products with different complex structures, regulations, requirements, and customers. Financial institutions are required to provide the highest-quality services and minimal risk. Thus, the applications of statistical learning in general and deep learning in particular vary among these services. We review machine learning techniques in the financial applications. The literature review divided into several subsection: 1- Machine Learning for Pricing Derivatives Instruments, 2- Machine Learning in Credit Risk, 3- Machine Learning for Financial Assets Cross-correlation, 4- Machine Learning for Prediction of Financial Time Series, 5- Machine Learning in Financial Sentiment Analysis. For a more in-depth recent survey on predicting stock prices, the reader may refer to Hu, Zhao, & Khushi (2021).

Our research focus is on the prediction of stock financial market prices. Most of the literature deals with stock prices as a time series, deducting the cross-correlation of financial assets or attempts to capture the effect of sentiments and news on the market; however, we look at the problem differently. We contribute to the literature by modeling the reaction to the price shock using deep learning.

2.1. Machine learning for pricing derivatives instruments

One of the financial applications is derivative pricing, which includes both options pricing and hedging; this research area has been explored using neural networks since 1990 (Ruf & Wang, 2019). For example, Das & Padhy (2017) proposed a hybrid model that combines parametric and nonparametric approaches to pricing European-style options. The parametric approach involves methods such as Black–Scholes, Monte Carlo simulation, and the finite difference method. In contrast, the nonparametric approach involves machine learning techniques such as support vector regression and extreme learning machine-based regression models. The aim is to boost investment decisions by accurately predicting option prices. The model validity was tested for the next next-day perdition of option prices and was compared to standard Black–Scholes and Monte Carlo simulations. The results showed a better performance for the proposed model compared to the benchmark. In addition, Culkin & Das (2017) trained a fully connected feed-forward deep learning neural network to reproduce the Black & Scholes (1973) option pricing formula. The results suggest that deep learning can be utilized for high-accuracy option pricing.

Additional applications may be seen in the work of Cao, Chen, & Hull (2019), in which they aimed at a better understanding of the volatility surface by analyzing the relationship between the S&P 500 and the changes in the implied volatility using a neural network. For a more extensive survey, the reader may refer to Bennell & Sutcliffe (2004); Chen & Sutcliffe (2012); Hahn (2013); Ruf & Wang (2019).

Beside the financial instruments, computational algorithms are used for technology pricing. Kar & Rakshit (2015) integrate fuzzy set theory and analytic hierarchy process for group decision making. The model aims to find a flexible pricing approach for IT services such as cloud computing, storage, and application resources. The model prioritizes and aggregates the key features and maps it as a utility function to estimate the trade-offs from each value driver.

2.2. Machine learning in credit risk

Another application of machine learning in finance is managing credit risk. Roy et al. (2018) utilized deep learning to detect credit card fraud. Different network structures with various parameters were explored on a data set of 80 million credit card transactions. A sensitivity analysis of the model parameters regarding fraud detection capability was provided; moreover, Luo, Wu, & Wu (2017) evaluated corporate credit scoring applied to credit default swaps using a deep belief network with restricted Boltzmann machines. The proposed model was compared to logistic regression, a multilayer perceptron (MLP), and a support vector machine. The results showed the superiority of the deep belief network over the other methods. In addition, Zhang & Trubey (2019) employed Bayesian logistic regression, decision trees, a random forest, a SVM, and an artificial neural network (ANN) on actual transaction data from US financial institutions. The algorithms were combined with sampling techniques to increase the number of laundering events. The results were promising, which indicates that machine-learning techniques can efficiently detect money laundering events.

2.3. Machine learning for financial assets cross-correlation

Machine learning techniques have been used to discover the cross-correlation and causality among financial instruments. Methodologies such as collaborative filtering, DNN, and LSTD can support decisions by dedicating financial assets' similarity and effect on the others.

Al-Sulaiman & Al-Matouq (2021) developed a collaborative filtering framework to build a recommendation system for extracting arbitrage opportunities by taking advantage of the time lag in world market indices. This theoretical framework involves developing user-based collaborative filtering, singular value decomposition imputes, and matrix

completion via convex nuclear norm minimization. The proposed methods were compared to other trading strategies, such as momentum, reverse momentum, naive random, and minimum variance strategies. The experimental results showed the consistently higher performance of nuclear norm minimization collaborative filtering over the other strategies.

Lee & Yoo (2019) studied the cross-correlation between the South Korean and US stock markets using a multimodal deep learning approach motivated by the empirical evidence of the effect of the US closing price on the South Korean stock market and vice versa. They used the KOSPI index as a proxy for the South Korean market and the S&P 500, DJ, and NASDAQ as proxies for the US market, where they downloaded the daily trades for 2006–2017. The data were then split into 40% training, 30% validation, and 30% testing sets. A set of five inputs was considered for training the network, namely, the high, open, low, overnight close, and overnight open-to-close returns for the KOSPI, S&P 500, DJ, and NASDAQ. The aim was to predict the daily close-to-close return of the KOSPI at time $t+1$. They implemented a single model and a multimodel deep neural network (DNN). Using the single model, they examined the results for KOSI-DNN, S&P 500-DNN, DJ-DNN, and NASDAQ-DNN separately. However, in the multimodel, they joined the input features of all indices together into one DNN. The results indicated that the multimodel performance is much more accurate at prediction than the single model because of its ability to capture cross-correlations between the markets' indices.

2.4. Machine learning for prediction of financial time series

One of the most common applications of machine-learning techniques concerns the prediction of financial time series. For example, Kim (2003) utilized a support vector machine (SVM) to predict financial time series and compared its performance with those of backpropagation neural networks and case-based reasoning. He claimed that SVM provides promising results. Kim & Han (2000) implemented ANN to predict the stock price index. However, they optimized the weights of the neurons using a genetic algorithm to overcome the global search algorithm's limitations when using gradient descent. The experiment showed that the proposed training method could provide a 10% higher prediction accuracy than global search.

Lu, Li, Wang, & Qin (2021) combined convolution neural network (CNN) along with bi-directional long short-term memory network and attention mechanism to predict next-day stock price. The CNN is responsible for extracting the input data, while the BiLSTM forecasts the next-day closing price by utilizing the extracted data from CNN. In addition, AM measures the effect of the feature states on the past to improve the prediction accuracy. They compared the model's effectiveness with seven different methods on 1000 trading days on the Shanghai Composite Index. The model was the lowest in terms of MAE and RMSE and the highest in terms of R-squared.

Furthermore, Pai & Lin (2005) proposed a hybrid method that combined the autoregressive integrated moving average with SVM to predict stock time series. The experiment showed high potential using the proposed method. Furthermore, Enke & Thawornwong (2005) addressed the importance of which variables are fed into ANN for the accuracy of prediction. They implemented their experiments using 34 economic and financial input variables, including closing prices, dividends, dividend yields, and *T*-bills of varying maturity. The experiment showed that greater profits could be obtained by considering adequate variables compared to a buy-and-hold strategy.

Nabipour, Nayyeri, Jabani, Shahab, & Mosavi (2020) compared various machine learning methods for prediction accuracy on several financial assets. The proposed algorithms included a decision tree, a forest, adaptive boosting, extreme gradient boosting, SVM, naive Bayes, *K*-nearest neighbors, logistic regression, ANN, RNN, and long short-term memory (LSTM). The input that fed into these algorithms consisted of signals extracted from ten technical indicators of ten years of historical daily time series. The algorithms were evaluated using the metrics of accuracy and the area under the curve of the receiver operating characteristic. The evaluation of the experiments showed that the best performance results were those of the RNN and LSTM.

Ghosh & Sanyal (2021) provided a study to introspect the predictability of market fear. They have used the India volatility index (VIX) and 20-day rolling standard deviation of NIFTY returns to account for implied and historical volatility in the time of COVID. The significant features are selected using the Boruta feature selection methodology on features extracted from macroeconomic constructs, technical indicators, and Google search volume index on meaningful keywords. Gradient Boosting, Extra Tree Regression, DNN, and LTSM. The findings suggest that India VIX and historical volatility can be predicted using the proposed algorithms despite exhibiting a high degree of volatile traits.

Additionally, Chen, Jiang, Zhang, & Chen (2021) used a CNN to incorporate market information along with stock information to predict a stock's trend by transforming the collected information into an image. More specifically, they used improved graphs and dual convolutional neural networks to capture the market and stock features. The experiments were implemented on six randomly chosen Chinese stocks, and the results showed the superior performance of the algorithm compared to a benchmark of seven other machine learning algorithms.

Furthermore, Wu, Li, Herencsar, Vo, & Lin (2021) combined a CNN and LSTM to predict stock movements as a financial time series. The algorithm uses the stock's historical data and its derivatives to construct a sequence of arrays that are used as input images in the CNN. The output feature vectors of the CNN are used as input vectors in the LSTM. The experiments with the developed model were performed on various US and Taiwan stocks. The results showed the higher accuracy of the model compared with other models such as the CNN, SVM, and ANN.

2.5. Machine learning in financial sentiment analysis

The sentiment analysis heavily invests in machine learning to drive collective opinions on various subjects. The theory of sentiment analysis encompasses various disciplines such as political science, social and economic sciences. For example, Neogi, Garg, Mishra, & Dwivedi (2021) studied the international sentiments on Twitter to understand what drives the protests. The analysis was performed on 20,000 tweets on the protest using the bag of words model and term frequency-inverse document frequency through the utilization of Naive Bayes, decision tree random forest, and SVM. They conclude that combining the random forest with the bag of words model provides the highest accuracy for classification. In the tourism industry, Mishra, Urolagin et al. (2019) used term frequency-inverse document frequency and Cosin similarity to provide more options of hotels based on tourist reviews. Similarly, Similarly, Mishra, Urolagin, & Jothi (2020) developed a point of an interest recommender system using a sentiment intensity analyzer that weights the reviews in order to suggest top tourist locations for the visitor. Furthermore, Asghar et al. (2021) analyze customer feedback on Twitter by developing sentiment-based eSystem using hybridized fuzzy and DNN to measure customer satisfaction. The proposed model uses LSTM to predict positive and negative sentiment polarity. The LTSM followed by fuzzy logic that determined satisfaction level of the customers. The results shows 92.86% accuracy outperforming the benchmark approaches.

Maqsood et al. (2020) used LR, SVM, and deep learning to predict stock prices in developed, emerging, and underdeveloped markets. The experiments were performed on selected stocks from markets in the US, Hong Kong, Turkey, and Pakistan. This method considers the effect of significant events by measuring sentiment on the platform Twitter from 2012 to 2016. The significant events involved the US (2012) and Mexican (2012) elections, attacks in Ghaza (2014), Brexit (2016), refugee migration (2015), Hong Kong protests (2014), a hijacked plane in Cyprus (2016), and the Lahore blast (2016). They evaluated the performance using the mean absolute error and mean square error, and the experiments showed improvement when sentiment was included.

Hagenau, Liebmann, & Neumann (2013) utilized text mining techniques to extract useful information from the financial news. The objective is to increase stock prediction accuracy using robust feature selection given exogenous market feedback. The results show that robust feature selection would augment the machine learning algorithm outputs and reduce the overfitting problem.

Huang & Liu (2020) utilized logistic regression to establish a basic stock price prediction model. The logistic regression model is integrated with a sentiment scores engine quantified using text mining techniques on social media opinions and stock-related news. The results show an improvement in the prediction of stock prices and thus can be used as a reference for investment strategies. Similarly, Huang, Zaeem, & Barber (2019) argue its equivalent failing of ignoring the information of social and trusting everybody on social media. Thus, they analyzed social media information to filter its credibility and trustworthiness. The analysis was accomplished by establishing trust scores to filter the tweets related to the stock market.

Nguyen, Shirai, & Velcin (2015) focused on the sentiment analysis of specific topics to develop a stock prediction model. The authors proposed a joint sentiment/topic model and aspect-based Sentiment model for the sentiment analysis. The models' effectiveness was evaluated by comparing the accuracy of 18 stocks in a one-year transaction. The comparison was performed against baseline models of the price-only method, human sentiment, sentiment classification, and latent Dirichlet allocation methods. The results showed an extra accuracy of 2.07% comparing baseline models.

Asghar et al. (2021) attempt to measure the predictability of market fear in terms of implied and historical volatility of equity markets. They implemented the study on the India VIX and 20-day rolling standard deviation of NIFTY returns to account for implied and historical volatility, respectively, during the COVID-19 timeline. The model inputs feature consists of pertinent macroeconomic constructs, technical indicators, and volume of meaningful keywords search on Google. The predictive model was constructed using diverse machine learning algorithms, including Gradient Boosting Extra Tree Regression, DNN, and LSTM. The findings suggest that India VIX and historical volatility can be effectively predicted utilizing the proposed architectures.

Tandon, Revankar, & Parihar (2021) investigates the correlation between the excited tweets and the prices of cryptocurrencies. In addition, they attempt to predict the future price values of Bitcoin using its past importance using cryptocurrencies financial data, Twitter data, RAPIDS, and cuml. The price forecasting was done using augmented dickey fuller test and ARIMA models; 10 future values of bitcoin were predicted with 96% accuracy and 0.0395 average error.

3. Methodology

In the methodology section we show the computational approach, data collection and robustness measure of the research.

3.1. Computational approach

Deep learning is classified as a subset of machine learning, which is classified as a subset of artificial intelligence (Deng, Yu et al., 2014). A common method of implementing deep learning is using an ANN with multiple hidden layers. The ANN mimics a biological system of neurons to process large amounts of data. Common types of ANN are deep feed-forward neural network (DFNN), convolutional recurrent neural, and LSTM networks. The employment of the ANN types depends on the problem. For example, the convolutional neural network may be suitable for image processing, and text mining problems, such as the work of Anand, Urolagin, & Mishra (2021) where they used the convolutional neural network to recognize hand gestures from a video or image input. In addition, Gadekallu et al. (2021) implement a crow search algorithm using a convolutional neural network for Hand gesture classification. The recurrent and LTSM may be suitable for time series problems such as Nabipour et al. (2020), or the problems that require a bidirectional flow of information, such as measuring customer satisfaction such as Asghar et al. (2021).

This paper aims to measure a reaction for an event in one-way traffic of information flow. The event is represented as a stock price sharp movie, and the response is the stock price in the future. Thus, we approach the problem through the DFNN. The DFNN consists of multiple input nodes that represent the data of interest. These nodes pass data to the processing elements that sum the weights multiplied by the associated input data with the biases multiplied by their weights. Fig. 1 shows an illustration of an ANN and Table 1 describes the notation used in this research. The processing elements from the inputs to the first hidden layer are shown in Eq. (1).

$$F(S_j) = \sum_{i=1}^{n} w_{ki,j} x_{ki,j} + \sum_{i=1}^{n} w_{B_{k,j,i}} x_{k,i,j} \quad \forall j \tag{1}$$

where $x_{k,i,j}$ represents the input/node i in layer k that is connected to node j in the successive layer $k+1$, $w_{ki,j}$ represents the weight associated with the input/node i in the layer k connected to node j in the layer $k+1$.

The linear combination generated by the processing element $F(S_j)$ is then used as an input to an activation function. The activation function takes the weighted sum of the input and the bias as its input to determine whether this sum should be output. The activation function works as a threshold to activate a specific node. It helps normalize the outputs from processing element $F(S_j)$ to a range of either -1–1 or 0–1; this is an essential element in the deep learning neural network. Careful selection of the activation function expedites the convergence of the neural net. Failure to select the appropriate activation function may decelerate the convergence process or even stop the network from converging at all.

Activation functions can be step functions, linear functions, or nonlinear functions. A step function is a binary function that generates either zero or one when the input value is below or above a certain threshold, respectively. A linear function can manage multiple outputs but stops the multiple-layer effect because a combination of linear functions for the layers is still a linear function. Furthermore, we cannot optimize the weights in a neural network using backpropagation since this requires deriving the gradient descent, which results in a constant. A linear function turns a neural network into a linear regression.

Nonlinear functions make deep learning a powerful tool because they capture the complex nonlinear relationship between inputs and outputs. The sigmoid function is a popular function with an S-shaped curve that returns real values between 0 and 1; it has the problem that it tends to be flat for large negative or positive input values, which causes a vanishing gradient that prevents learning the neural network. The sigmoid function takes only positive values, which may often be a flaw. The hyperbolic tangent function scales the sigmoid to generate real numbers between −1 and 1; however, it shares the vanishing gradient problem with the sigmoid function.

The rectified linear unit (ReLU) function is an activation function that was mathematically introduced and supported by Hahnloser & Seung (2001). Glorot, Bordes, & Bengio (2011) were the first to claim that the ReLU can provide superior results when used to train an NNT. Since then, it has been proven to improve NNTs in many cases. It is the most commonly used activation function in designing deep learning today. It has the advantage of not activating all neurons simultaneously. In ReLU, only positive inputs are activated, while negative values are converted to zero, and neurons are deactivated. This property helps develop the NNT, makes it progressively proficient, and addresses the vanishing gradient problem. The ReLU can be defined as a function that produces a minimum value of 0.

$$x_{k+1,l,m} = \max(0, F(S_j)) \tag{2}$$

where $x_{k+1,l,m}$ used as input of node l in layer $k+1$ that connected to node m in the succeeded layer.

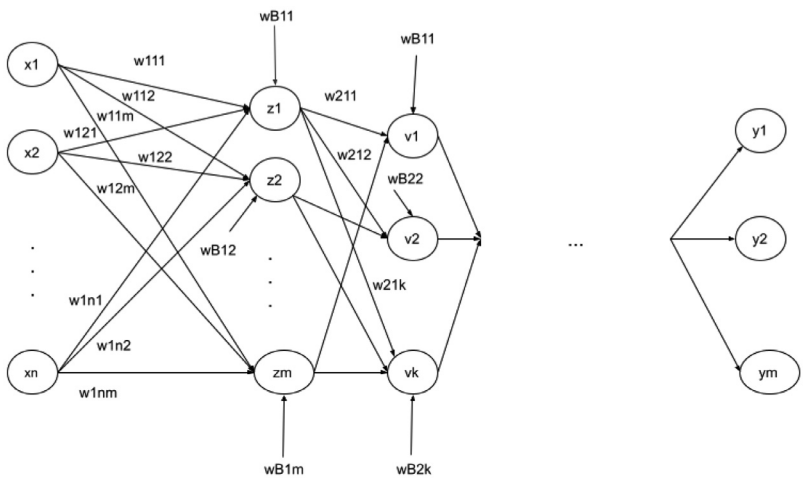

Fig. 1. Feed-forward neural network.

Processing Element (input-hidden layer 1):

$$F(S_j) = \sum_{i=1}^{n} w_{1ji} x_i + \sum_{i=1}^{n} w_{Bji} x_i \quad \forall j$$

Table 1
Notations.

Notation	Description
n	total number of nodes in the neural network layer
k	index for layer number in the neural network
i	index for the input/node number in the neural network
j	index for the node in the successive layer $k+1$ in theneural network
$x_{k,i,j}$	input/node i in layer k that is connected to node j in the successive layer $k+1$
$w_{ki,j}$	weight associated with the input/node i in the layer k connected to node j in the layer $k+1$
$F(S_j)$	linear combination generated from the processing element
$x_{k+1,l,m}$	input of node l in layer $k+1$ that connected to node m in the succeeded layer.
α	a small, non-negative slope coefficient in Leaky ReLU activation function
MAE	mean absolute error
N	sample of size of stocks
$\hat{p}_h$	predicted price of stock h where $h = 1, 2, \ldots, N$
p_h	actual price of stock h where $h = 1, 2, \ldots, N$
p_t	daily closing prices
r_t	daily return
v_t	trading volume
δ	threshold defines the large movement
L_l	denoting the number of hidden layers

The problem with ReLU is that it is impossible to differentiate the 0 values when we aim to optimize the weights $\boldsymbol{w}$ in the network. Therefore, the leaky ReLU function (Maas, Hannun, & Ng, 2013) can serve as a potential substitute for ReLU to overcome this problem. The leaky ReLU LReLU function contains a small, nonnegative slope coefficient that addresses the differentiation problem of zero values. Fig. 2 shows the ReLU and leaky ReLU functions with $\alpha = 0.01$.

$$x_{k+1,l,m} = \begin{cases} F(S_j) & \text{if } F(S_j) > 0 \\ \alpha F(S_j) & \text{if } F(S_j) < 0 \end{cases} \tag{3}$$

After calculating the neural network's feed-forward outputs, we supervised the learning method by observing the mean of either the absolute error or the square error. In this paper, we aim to minimize the mean of the absolute error.

$$MAE = \frac{1}{N} |p_h - \hat{p}_h| \tag{4}$$

where $\hat{p}_i$ and p_i are the predicted prices of sample i and n is the sample size.

Several optimization methods have been developed, and many of them are based on a gradient descent method, such as backpropagation (Hecht-Nielsen, 1992), resilient propagation and Manhattan propagation. Other techniques rely on numerical methods such as the genetic algorithm and simulated annealing to train a network. In 2012, Tieleman & Hinton (2012) proposed root-mean-square propagation to optimize the weights in a network.

A robust algorithm is an algorithm that has low bias and variance. The low bias shows the ability of accurate prediction on the training set, and low variance shows the power of consistent prediction on the testing data set. A network's structure is essential for providing satisfactory results that support the required goal. Determining the proper structure will ensure good performance and avoid overfitting.

However, several elements may affect the structure when we design a DNN. These elements include selecting the numbers of hidden layers and contained nodes in each layer. In addition, they include determining the number and nature of the input nodes. Further elements include the activation function, training method, and training protocol. Often, it is not an easy task to determine the configuration of the NNT, and the determination of the structure, which is a difficult task, can be accomplished through experimental analysis.

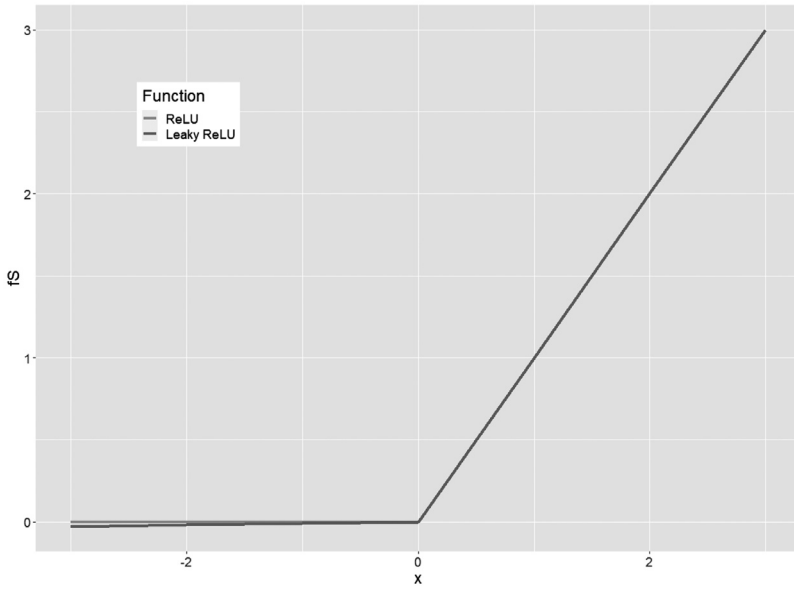

Fig. 2. ReLU and Leaky ReLU functions.

Table 2
Closing price, returns and volumes of stocks in the S&P 500.

No.	1			2			...	498		
Ticker	MMM			ABT			...	ZTS		
Date	closing price	return	volume	closing price	return	volume	...	closing price	return	volume
12/31/2018	190.54	NA	1,804,400	69.5	NA	8,737,200	...	84.99833	NA	1,485,200
1/2/2019	190.95	0.00215	2,475,200	66.22	−0.0391	10,570,700	...	83.94504	−0.01239	2,665,600
1/3/2019	183.76	−0.0376	3,358,200	68.11	−0.0471	8,299,100	...	81.04353	−0.03456	2,390,900
.	.	.	.	.	.	.		.	.	.
.	.	.	.	.	.	.	...	.	.	.
.	.	.	.	.	.	.		.	.	.
12/27/2019	177.26	0.00379	1,472,900	87.4	0.001375	2,508,100	...	133.25	0.001654	1,296,100

3.2. Data collection

The daily closing prices p_t, returns r_t and trading volume v_t for all stocks in the S&P 500 were downloaded for the period from Dec 31, 2018, to Dec 27, 2019. The data was downloaded using "BatchGetSymbols" package in R (Perlin, 2020) (Table 2).

The input attributes are the closing prices, p_t, returns r_t and trading volume v_t. The output is the closing prices after some period of time.

However, the data is filtered for each experiment. For example, for some of the experiments we take the closing prices, returns and volume only if the return is more than 2% or less than −2%. The pre-described positive and negative 2% count for the large movements.

The large movements of stocks are defined by a threshold δ, and we set $\delta_1 = \pm 2\%$, $\delta_2 = \pm 4\%$ and $\delta_3 = \pm 8\%$. For δ_1, a movement is considered large if the stock return is above or below 2% on any given date; similar conditions apply for δ_2 and δ_3. These conditions bring the number of sampled data values to 19,273 for δ_1, 3304 for δ_2 and 370 for δ_3.

Next, we explored the statistics of the collected data by measuring the average returns for the training and testing data: 0.000637 and 0.00115 for δ_1, −0.00116 and −0.00338 for δ_2 and 0.001149 and −0.0015 for δ_3. The percentages of positive returns given the threshold δ_i are 52.3%, 48.3% and 51.6% for δ_1, δ_2 and δ_3, respectively. However, the percentages of negative returns for δ_1, δ_2 and δ_3 are 47.7%, 51.6% and 48.3%, respectively. These statistics show a balance of positive and negative returns for the given threshold in the stocks included in the S&P 500 index.

4. Results

In this section we show the robustness of the proposed method by conducting various experiments that produce different mean absolute error based on the predetermined experiment configurations. The section is divided into two subsections: experimental analysis of the design neural network structure and comparison with other algorithms.

4.1. Experimental analysis for model robustness

The independent variables are p_t, r_t and v_t for any given date t. The aim of the study is to predict the reaction to large movements after one day, two days, 10 days, one month, and two months. Thus, the dependent variable for each experiment is the price at different time steps, i.e., p_{t+1}, p_{t+2}, p_{t+10}, p_{t+30} and p_{t+60}.

We explored the performance of the DFNN for predicting the reaction to large movements in a given period by measuring the absolute difference between the predicted price and the actual price as in Eq. (4).

Various structures of NNT were explored. Denoting the number of hidden layers by L_l, the network structures may contain $L_1 = 3$ hidden layers, $L_2 = 4$ hidden layers or $L_3 = 5$ hidden layers. The numbers of nodes n for each layer of $L_1 = 3$ are $n_1 = 20$, $n_2 = 10$ and $n_3 = 5$. For $L_2 = 4$, the numbers of nodes are $n_1 = 40$, $n_2 = 20$, $n_3 = 10$ and $n_4 = 5$. The numbers of nodes when $L_3 = 6$ are distributed as follows: $n_1 = 80$, $n_2 = 40$, $n_3 = 20$, $n_4 = 10$, and $n_5 = 5$.

Combining these factors creates a factorial design with the first factor containing five levels; the second and third factors contain three levels, bringing the total number of experiments to $5 \times 2^3 = 45$. Each experiment was replicated five times, for a total of 225 runs. Fig. 3 shows two of the structures developed in the experiments. The performance of each experiment was measured using the mean absolute error as in 4.

However, a large neural network structure contains a large number of parameters (weights and biases), leading to overfitting. The problem of overfitting can be overcome by the regularization or dropout concepts (Srivastava, 2013; Srivastava, Hinton, Krizhevsky, Sutskever, & Salakhutdinov, 2014). The dropout concept is based on the dropout coefficient $0 \le d \le 1$. Instead of encompassing all the input features and

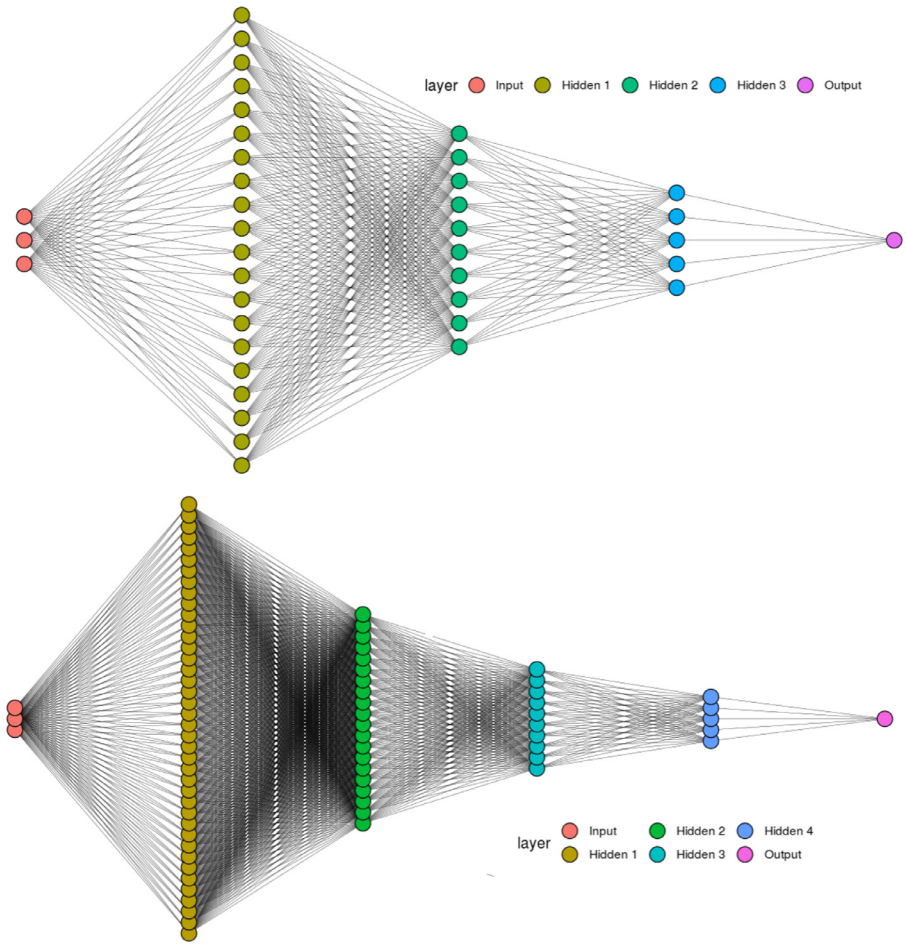

Fig. 3. Structure of neural network with three layer (top) and four layer (bottom).

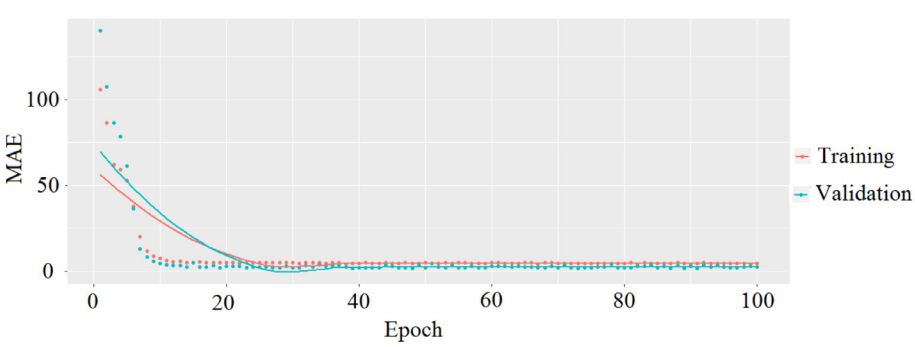

Fig. 4. Sample performance of an experiment.

the activation functions in the hidden layers, the coefficient d allows us to single out a subset of features in the input layer and activation functions in the hidden layers. In our experiments, the drop-out coefficient is $d = 0.01$ in all layers. The experiments were implemented using the R statistical package with various libraries, such as Keras (Allaire & Chollet, 2019), neuralnet (Fritsch, Guenther, & Wright, 2019), and TensorFlow (Allaire & Tang, 2019).

Fig. 4 shows the performance of a sample run. The red dots in the figure represent the mean absolute error over the epochs, while the light blue dots represent the validation loss of the mean absolute error. An overfitting indication occurs if the mean absolute error of the training set (red dots) crosses below the mean absolute error validation loss (light blue dots). Table 3 shows the results of the experiments.

We analyzed the results using two-way ANOVA to test for significant differences in the absolute mean error. The ANOVA results showed that there is a significant difference when we consider different lags in prediction (p_{t+1}, p_{t+2}, p_{t+10}, p_{t+30}, and p_{t+60}). Additional significant differences were found at different threshold levels ($\delta_1 = \pm 2\%$, $\delta_2 = \pm 4\%$ and $\delta_3 = \pm 8\%$) and at different levels of the hidden layers ($L_1 = 3$, $L_2 = 4$ and $L_3 = 5$). The p-values $< 2e - 16$ for all factors. The ANOVA table and box plots of the analysis are shown in Table 4 and Fig. 5.

Furthermore, Tukey's range test was conducted. The results of the test showed that regarding the main factor effect of prediction lag, deep learning is more accurate when predicting stock prices in a shorter time period (p_{t+1} and p_{t+2}) than when predicting stock prices over a longer period of time. In addition, deep learning shows better performance when

Table 3
Experimental results.

EX	p_{t+i}	δ	L	MAE_1	MAE_2	MAE_3	MAE_4	MAE_5	Mean
1	1	0.02	3	4.16	2.37	4.96	5.81	3.50	4.16
2	1	0.02	4	1.09	0.86	1.60	1.66	1.09	1.26
3	1	0.02	5	0.81	0.71	0.64	0.81	0.92	0.78
4	1	0.04	3	5.49	5.21	2.73	3.74	3.95	4.23
5	1	0.04	4	1.33	3.94	2.31	3.45	1.41	2.49
6	1	0.04	5	1.74	2.76	2.11	2.16	2.67	2.29
7	1	0.08	3	13.34	15.30	15.58	10.52	15.65	14.08
8	1	0.08	4	12.55	9.35	9.15	9.18	5.11	9.07
9	1	0.08	5	6.39	5.29	6.08	7.25	6.94	6.39
10	2	0.02	3	4.03	5.57	7.78	5.41	6.01	5.76
11	2	0.02	4	2.85	2.65	3.20	2.54	2.75	2.80
12	2	0.02	5	2.69	2.71	3.04	2.82	2.64	2.78
13	2	0.04	3	3.35	5.25	4.13	4.00	4.18	4.18
14	2	0.04	4	4.75	3.18	2.89	3.74	3.10	3.53
15	2	0.04	5	3.43	3.98	3.34	3.36	3.37	3.50
16	2	0.08	3	12.74	13.43	11.08	13.79	11.18	12.45
17	2	0.08	4	7.22	10.78	9.23	10.97	9.42	9.53
18	2	0.08	5	5.01	5.44	8.40	5.87	6.18	6.18
19	10	0.02	3	69.94	85.55	66.40	69.83	78.32	74.01
20	10	0.02	4	83.08	67.79	60.70	61.71	71.40	68.94
21	10	0.02	5	63.07	63.03	60.28	64.99	63.08	62.89
22	10	0.04	3	18.01	17.63	18.28	16.75	16.98	17.53
23	10	0.04	4	16.49	17.41	16.64	16.25	16.07	16.57
24	10	0.04	5	15.74	15.76	16.67	16.00	15.90	16.01
25	10	0.08	3	16.29	16.21	16.06	16.52	16.09	16.23
26	10	0.08	4	19.21	15.00	13.26	12.93	14.72	15.03
27	10	0.08	5	9.48	12.55	11.01	10.04	11.50	10.92
28	30	0.02	3	121.63	136.24	98.56	118.81	114.85	118.02
29	30	0.02	4	102.41	106.71	101.10	99.49	102.42	102.42
30	30	0.02	5	114.08	102.06	97.94	98.49	97.45	102.00
31	30	0.04	3	105.07	98.70	113.29	108.16	107.09	106.46
32	30	0.04	4	100.33	101.43	99.39	97.90	115.41	102.89
33	30	0.04	5	94.28	92.23	93.21	91.22	89.97	92.18
34	30	0.08	3	51.67	52.77	47.64	51.92	52.57	51.31
35	30	0.08	4	34.93	36.37	36.22	36.94	36.80	36.25
36	30	0.08	5	35.03	34.83	34.44	35.79	35.14	35.05
37	60	0.02	3	110.94	110.90	110.93	110.88	110.44	110.82
38	60	0.02	4	101.84	86.07	110.51	104.99	108.14	102.31
39	60	0.02	5	77.51	85.40	85.66	82.99	88.69	84.05
40	60	0.04	3	85.06	85.1	83.82	85.97	85.73	85.14
41	60	0.04	4	87.32	84.62	82.23	83.08	83.46	84.14
42	60	0.04	5	93.46	79.19	77.19	75.17	74.76	79.95
43	60	0.08	3	58.00	57.29	58.24	58.58	55.68	57.56
44	60	0.08	4	52.73	52.08	52.94	53.25	50.62	52.33
45	60	0.08	5	41.43	41.34	41.73	41.01	42.42	41.59

Table 4
ANOVA table.

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
p_{t+i}	4	256,788	64,197	4221.314	<2e−16
δ	2	36,703	18,351	1206.714	<2e−16
L	2	3059	1530	100.578	<2e−16
$p_{t+i} : \delta$	8	51,136	6392	420.313	<2e−16
$p_{t+i} : L$	8	1397	175	11.486	4.05e−13
$\delta : L$	4	8813	99	6.521	6.36e−05
$p_{t+i} : \delta : L$	16	397	682	2.803	0.00046
Residuals	180	682	2737		

Table 5
Tukey's range test.

	Difference	Lower	Upper	P-value
$p_{t+10} - p_{t+1}$	28.15	25.89	30.42	0
$p_{t+2} - p_{t+1}$	0.663	−1.60	2.92	0.928
$p_{t+30} - p_{t+1}$	77.98	75.71	80.25	0
$p_{t+60} - p_{t+1}$	−27.49	70.30	74.83	0
$p_{t+2} - p_{t+10}$	−27.49	−29.75	−25.22	0
$p_{t+30} - p_{t+10}$	49.82	47.56	52.09	0
$p_{t+60} - p_{t+10}$	44.41	42.15	46.68	0
$p_{t+30} - p_{t+2}$	77.32	75.05	79.58	0
$p_{t+60} - p_{t+2}$	71.90	69.64	74.17	0
$p_{t+60} - p_{t+30}$	−5.41	−7.67	−3.14	0
$\delta_2 = 0.04 - \delta_1 = 0.02$	−14.79	−16.29	−13.28	0
$\delta_3 = 0.08 - \delta_1 = 0.02$	−16.47	−32.77	−29.76	0
$\delta_3 = 0.08 - \delta_2 = 0.04$	−3.68627	−17.98	−14.97	0
$L_2 = 4 - L_1 = 3$	−4.82	−6.32	−3.32	0
$L_3 = 5 - L_1 = 3$	−9.02	−10.52	−7.51	0
$L_3 = 5 - L_2 = 4$	−10.1305	−4.19	−2.69	0

the threshold is strict ($\delta_1 = \pm 2\%$), with a significant difference compared to the other thresholds. This may be because the stocks reveal a clear pattern under this threshold or because more training data are available.

As expected, the deep learning performance is boosted when we add more layers, with a significant difference. This comes with a greater computational time. The Tukey's range test results are shown in Table 5.

The findings are summarized as follows:

1. The DFNN performs better with a short-term horizon when shock is relatively low ($\delta = \pm 2\%$).
2. The DFNN performance improves for long-term horizon as the shock increases ($\delta = \pm 8\%$).

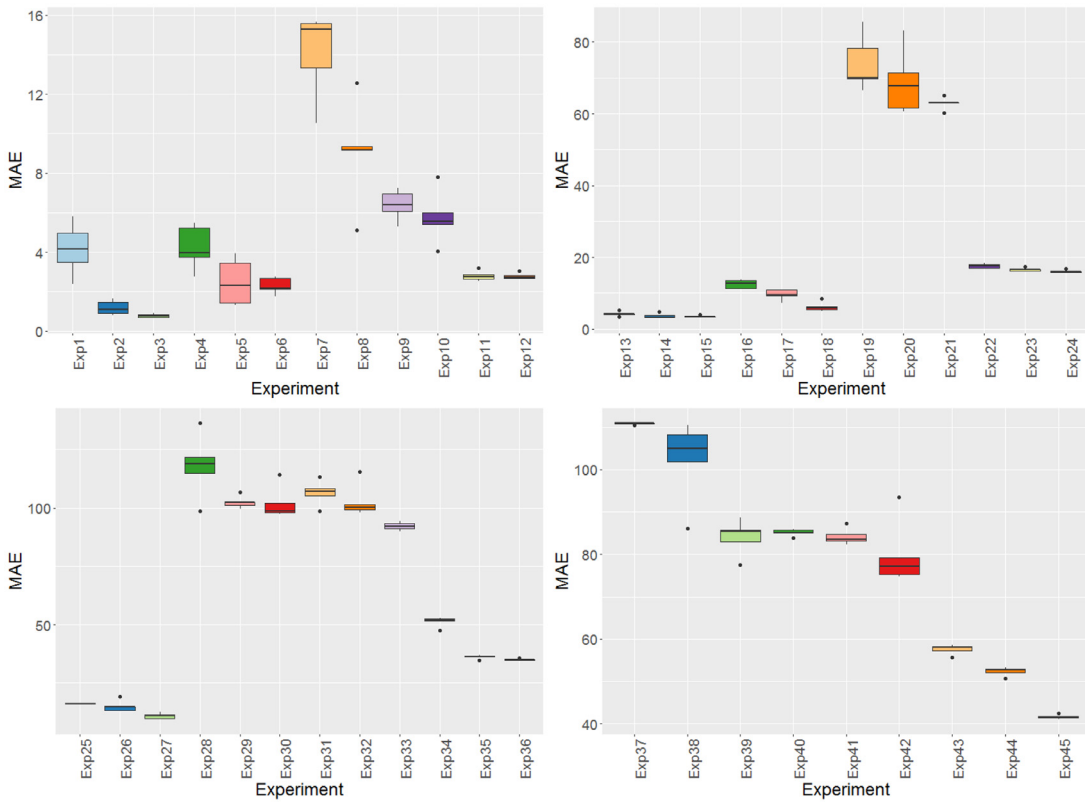

Fig. 5. Box plots of the experiments results .

3. In general, the accuracy of shorter-term predictions is higher than the long-term predictions.
4. Regardless computational and overfitting flaws, boosting hidden layers and neurons increases the accuracy.

The experiment's main observation is that the prediction of reactions to high anomalies are better in the longer term than in the shorter term. In comparison, the prediction of reactions to lower anomalies is more accurate in the shorter term. However, there is a better ability to predict short-term reactions, and the increase in the number of hidden layers increases the prediction accuracy.

4.2. Comparison with alternative algorithms

In this section, we provide bench-marking criteria to test the system's results. The contrast and validation of the experimental findings yielded by the proposed method is performed by trading approaches and algorithms employed on the same data sets and influencing factors. The alternative algorithms include linear regression (LR), MLP, SVM, and the RNN. The alternative algorithms are compared with the developed DFNN with five hidden layers. We predict the price one day ahead and 60 days ahead with $\delta_1 = \pm 2\%$ and $\delta_3 = \pm 8\%$. In the preceding experiments, the data were divided into 70% for training and 30% for testing. The proposed algorithms yield a low MAE compared to that of the benchmark algorithms. Table 6 shows the $MAEs$ of different approaches.

In addition, we compare the performance of different activation functions, such as linear, sigmoid, softmax and tanh functions, and the results are in favor of the rule function. Table 7 shows the comparisons.

Table 6
MAE of alternative approaches.

	DFNN	LR	MLP	SVM
$\delta_1 = \pm 2\%$ and p_{t+1}				
MAE	0.78	2.4	4.0	2.7
$\delta_1 = \pm 2\%$ and p_{t+60}				
MAE	84.05	86.3	92.1	94.7
$\delta_3 = \pm 8\%$ and p_{t+1}				
MAE	6.39	9.8	42.5	10.4
$\delta_3 = \pm 8\%$ and p_{t+60}				
MAE	41.59	45.1	149.9	112.5

Table 7
MAE of different activation function on DFNN.

	RuLe	linear	sigmoid	softmax	tanh
	$\delta_1 = \pm 2\%$ and p_{t+1}				
MAE	0.78	3.5	81.5	76.4	45
	$\delta_1 = \pm 2\%$ and p_{t+60}				
MAE		7.7	93.2	96.3	90
	$\delta_3 = \pm 8\%$ and p_{t+1}				
MAE	6.39	86.2	90.4	105.4	91.1
	$\delta_3 = \pm 8\%$ and p_{t+60}				
MAE	41.58	44.5	85.7	94	84.5

5. Discussion

In this study, we attempted to predict the future price condition on a current large movement using a feed-forward deep learning network. In this section, we discuss the implication for theory and practice. In addition, we discuss the limitation and future research directions.

5.1. Implications for theory

Trading decisions that are built based on algorithmic developments are dominating the financial markets. Many algorithms attempt to predict future prices based on previous movements, technical analysis, or fundamental analysis.

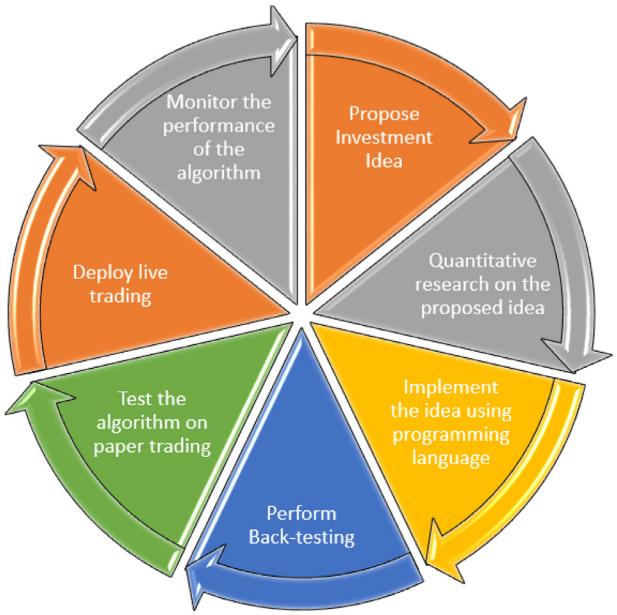

Fig. 6. cycle of algorithmic trading.

Here, we make a theoretical contribution in proposing a new model for predicting the future stock prices giving shock in the current prices. In this study, we have developed a feed-forward neural algorithm, which transfers the information of the stocks exposed to large movements as inputs to the network. The input information consists of the current closing price, return, and volume. Various structures of networks have been implemented with three, four, and five hidden layers. The data set was divided into 70% for training and 30% for testing, and the algorithm was trained using 100 epochs. LReLU was used as the activation function, and the algorithm was trained using root-mean-squared propagation.

The experiments were implemented on all stocks in the S&P 500 from Dec 2018 to Dec 2019. A factorial design was conducted with three factors, resulting in 45 experiments. The performance of the combinations was measured using the mean absolute error. The outputs were examined using variance analysis, and the investigations showed that deep learning had superior performance in shorter predictions of one or two days ahead. Furthermore, the performance tended to be accurate when the threshold was narrow. However, we should conduct further investigation to understand the reasons. Finally, suppose computational power is available. In this case, it is recommended to use multiple layers as long as the model does not overfit the data, as the results for multiple hidden layers tend to be superior.

5.2. Implications for practice

From a practical point, the developed methodology can be used to construct algorithmic trading systems that provide a buy or sell signal that is activated when shock occurs. The research represents the initial blocks building blocks of the algorithmic trading systems. To complete the cycle of the algorithmic trading systems, we need to implement and automate the research idea using one of the programming languages. Then we should perform back-testing and paper trading on the live stream for some time. After that, we need to deploy the live trading and monitor its performance for enactment. Fig. 6 shows cycle of algorithmic trading as presented by Al-Sulaiman (2021).

5.3. Limitations and future research directions

This study has certain limitations; first, we have considered a few input factors concentrated around the properties of the stock itself. Cross-correlation factors that examine the relationship with the market may be considered. Second, we have not incorporated the economic factors associated with the shock in the prices. Factors such as GDP and inflation rates may be taken into consideration.

Future extensions of this work may include adding more influencing factors to improve the deep learning algorithm's performance. Moreover, experimental studies may be conducted at different times and locations. It also should include testing the algorithm on different data sets of diverse time frames and other financial markets. A future extension may involve developing various trading strategies based on the algorithm's prediction to maximize the return of investments. In addition, given the availability of additional computational power, we would like to explore the optimal configuration of the DFNN in terms of accuracy, presuming the model is not overfitted. Namely, we would like to search the numbers of multiple hidden neural layers and hidden layer neurons with a greater iteration depth.

6. Conclusions

The applications of machine learning in finance are growing rapidly. They include fraud detection for credit cards, minimizing the risk of a portfolio, and predicting financial assets' future values. This paper hypothesizes that stocks' future prices may be accurately predicted using a few inputs, given that the stocks are exposed to a significant move in the negative or positive direction. We set thresholds to specify a great movement of the stock. The thresholds are set to be plus or minus two, four, and eight percentage points.

This research explores the possibility of predicting future stocks processes given significant changes in the current price. The hypothesis was tested by developing a deep forward neural network model and validated by implementing experiments with different configurations. The model robustness is examined using mean absolute error, and we treat the bias-variance problem by utilizing the regularization and dropout concepts. In addition, the proposed algorithm compared with benchmark machine algorithms, including linear regression, multiplayer perception, SVM, and recurrent neural network.

The results show that the model can predict future prices; however, it performs better in predicting short-term prices and when the current changes in prices are higher. Additionally, the number of hidden layer and hidden layer neurons improves the accuracy. Furthermore, the model shows higher robustness compared to benchmark algorithms.

Economic factors, such as GDP and inflation rates, may be included to boost the research output.

Acknowledgement

The author would like to provide a sincere thanks to the Advance Operations and Analytical Research Group (AOARG) at Prince Sultan University for the support and partial funding of the research.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.jjimei.2022.100071

References

Achelis, S. B. (2001). *Technical analysis from A to Z*. New York: McGraw Hill New York.
Al-Sulaiman, T. (2021). Algorithmic trading: Research and implementations.
Al-Sulaiman, T., & Al-Matouq, A. (2021). A convex collaborative filtering framework for global market return prediction. *IEEE Access, 9*, 29458–29469.
Allaire, J., & Chollet, F. (2019). keras: R interface to 'keras'. https://CRAN.R-project.org/package=keras R package version 2.2.5.0.
Allaire, J., & Tang, Y. (2019). tensorflow: R interface to 'tensorflow'. https://CRAN.R-project.org/package=tensorflow R package version 2.0.0
Alsulaiman, E. T. (2013). Classifying technical indicators using *K*-medoid clustering. *The Journal of Trading, 8*(2), 29–39.
Anand, K., Urolagin, S., & Mishra, R. K. (2021). How does hand gestures in videos impact social media engagement-insights based on deep learning. *International Journal of Information Management Data Insights, 1*(2), 100036.

Asghar, M. Z., Subhan, F., Ahmad, H., Khan, W. Z., Hakak, S., Gadekallu, T. R., et al. (2021). Senti-eSystem: A sentiment-based eSystem-using hybridized fuzzy and deep neural network for measuring customer satisfaction. *Software: Practice and Experience, 51*(3), 571–594.

Bennell, J., & Sutcliffe, C. (2004). Black–Scholes versus artificial neural networks in pricing FTSE 100 options. *Intelligent Systems in Accounting, Finance andManagement: International Journal, 12*(4), 243–260.

Black, F., & Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy, 81*(3), 637–654.

Cao, J., Chen, J., & Hull, J. C. (2019). A neural network approach to understanding implied volatility movements. Available at SSRN 3288067.

Chen, F., & Sutcliffe, C. (2012). Pricing and hedging short sterling options using neural networks. *Intelligent Systems in Accounting, Finance and Management, 19*(2), 128–149.

Chen, W., Jiang, M., Zhang, W.-G., & Chen, Z. (2021). A novel graph convolutional feature based convolutional neural network for stock trend prediction. *Information Sciences, 556*, 67–94.

Culkin, R., & Das, S. R. (2017). Machine learning in finance: The case of deep learning for option pricing. *Journal of Investment Management, 15*(4), 92–100.

Das, S. P., & Padhy, S. (2017). A new hybrid parametric and machine learning model with homogeneity hint for european-style index option pricing. *Neural Computing and Applications, 28*(12), 4061–4077.

Deng, L., Yu, D., et al. (2014). Deep learning: Methods and applications. *Foundations and Trends® in Signal Processing, 7*(3–4), 197–387.

Ehrhardt, M. C., & Brigham, E. F. (2011). *Financial management: Theory and practice*. Boston, Massachusetts: South-Western Cengage Learning.

Enke, D., & Thawornwong, S. (2005). The use of data mining and neural networks for forecasting stock market returns. *Expert Systems with Applications, 29*(4), 927–940.

Fama, E. F. (1991). Efficient capital markets: II. *The journal of finance, 46*(5), 1575–1617.

Fritsch, S., Guenther, F., & Wright, M. N. (2019). neuralnet: Training of neural networks. https://CRAN.R-project.org/package=neuralnet R package version 1.44.2

Gadekallu, T. R., Alazab, M., Kaluri, R., Maddikunta, P. K. R., Bhattacharya, S., Lakshmanna, K., et al. (2021). Hand gesture classification using a novel CNN-crow search algorithm. *Complex and Intelligent Systems*, 1–14.

Ghosh, I., & Sanyal, M. K. (2021). Introspecting predictability of market fear in indian context during COVID-19 pandemic: An integrated approach of applied predictive modelling and explainable AI. *International Journal of Information Management Data Insights, 1*(2), 100039.

Glorot, X., Bordes, A., & Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics* (pp. 315–323).

Hagenau, M., Liebmann, M., & Neumann, D. (2013). Automated news reading: Stock price prediction based on financial news using context-capturing features. *Decision Support Systems, 55*(3), 685–697.

Hahn, J. T. (2013). *Option pricing using artificial neural networks: An Australian perspective*. Princeton, New Jersey: Citeseer.

Hahnloser, R. H., & Seung, H. S. (2001). Permitted and forbidden sets in symmetric threshold-linear networks. In *Advances in neural information processing systems* (pp. 217–223).

Heaton, J., Polson, N. G., & Witte, J. H. (2016). Deep learning in finance. arXiv preprint arXiv:1602.06561.

Hecht-Nielsen, R. (1992). Theory of the backpropagation neural network. In *Neural networks for perception* (pp. 65–93). Amsterdam: Elsevier.

Hu, Z., Zhao, Y., & Khushi, M. (2021). A survey of Forex and stock price prediction using deep learning. *Applied System Innovation, 4*(1), 9.

Huang, J.-Y., & Liu, J.-H. (2020). Using social media mining technology to improve stock price forecast accuracy. *Journal of Forecasting, 39*(1), 104–116.

Huang, T.-C., Zaeem, R. N., & Barber, K. S. (2019). It is an equal failing to trust everybody and to trust nobody: Stock price prediction using trust filters and enhanced user sentiment on twitter. *ACM Transactions on Internet Technology (TOIT), 19*(4), 1–20.

Kar, A. K., & Dwivedi, Y. K. (2020). Theory building with big data-driven research–moving away from the "what" towards the "why". *International Journal of Information Management, 54*, 102205.

Kar, A. K., & Rakshit, A. (2015). Flexible pricing models for cloud computing based on group decision making under consensus. *Global Journal of Flexible Systems Management, 16*(2), 191–204.

Kim, K.-j. (2003). Financial time series forecasting using support vector machines. *Neurocomputing, 55*(1–2), 307–319.

Kim, K.-j., & Han, I. (2000). Genetic algorithms approach to feature discretization in artificial neural networks for the prediction of stock price index. *Expert Systems with Applications, 19*(2), 125–132.

Lee, S. I., & Yoo, S. J. (2019). Multimodal deep learning for finance: Integrating and forecasting international stock markets. arXiv preprint arXiv:1903.06478

Lu, W., Li, J., Wang, J., & Qin, L. (2021). A CNN-BiLSTM-AM method for stock price prediction. *Neural Computing and Applications, 33*(10), 4741–4753.

Luo, C., Wu, D., & Wu, D. (2017). A deep learning approach for credit scoring using credit default swaps. *Engineering Applications of Artificial Intelligence, 65*, 465–470.

Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013). Rectifier nonlinearities improve neural network acoustic models. In *Proc. ICML: vol. 30* (p. 3).

Maqsood, H., Mehmood, I., Maqsood, M., Yasir, M., Afzal, S., Aadil, F., et al. (2020). A local and global event sentiment based efficient stock exchange forecasting using deep learning. *International Journal of Information Management, 50*, 432–451.

Mishra, R. K., Urolagin, S., & Jothi, J. A. A. (2020). Sentiment analysis for poi recommender systems. In *2020 seventh international conference on information technology trends (ITT)* (pp. 174–179). IEEE.

Mishra, R. K., Urolagin, S., et al. (2019). A sentiment analysis-based hotel recommendation using TF-IDF approach. In *2019 international conference on computational intelligence and knowledge economy (ICCIKE)* (pp. 811–815). IEEE.

Nabipour, M., Nayyeri, P., Jabani, H., Shahab, S., & Mosavi, A. (2020). Predicting stock market trends using machine learning and deep learning algorithms via continuous and binary data; a comparative analysis. *IEEE Access, 8*, 150199–150212.

Neogi, A. S., Garg, K. A., Mishra, R. K., & Dwivedi, Y. K. (2021). Sentiment analysis and classification of indian farmers' protest using twitter data. *International Journal of Information Management Data Insights, 1*(2), 100019.

Nguyen, T. H., Shirai, K., & Velcin, J. (2015). Sentiment analysis on social media for stock movement prediction. *Expert Systems with Applications, 42*(24), 9603–9611.

Pai, P.-F., & Lin, C.-S. (2005). A hybrid ARIMA and support vector machines model in stock price forecasting. *Omega, 33*(6), 497–505.

Perlin, M. (2020). Batchgetsymbols: Downloads and organizes financial data for multiple tickers. R package version, vol. 2(7).

Roy, A., Sun, J., Mahoney, R., Alonzi, L., Adams, S., & Beling, P. (2018). Deep learning detecting fraud in credit card transactions. In *2018 systems and information engineering design symposium (SIEDS)* (pp. 129–134). IEEE.

Ruf, J., Wang, W. (2019). Neural networks for option pricing and hedging: Aliterature review. Available at SSRN 3486363.

Srivastava, N. (2013). *Improving neural networks with dropout* p. 7. University of Toronto.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research, 15*(1), 1929–1958.

Tandon, C., Revankar, S., & Parihar, S. S. (2021). How can we predict the impact of the social media messages on the value of cryptocurrency? Insights from big data analytics. *International Journal of Information Management Data Insights, 1*(2), 100035.

Tieleman, T., & Hinton, G. (2012). Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural Networks for Machine Learning, 4*(2), 26–31.

Wu, J. M.-T., Li, Z., Herencsar, N., Vo, B., & Lin, J. C.-W. (2021). A graph-based CNN-LSTM stock price prediction algorithm with leading indicators. *Multimedia Systems, 7*(4), 1–20.

Zhang, Y., & Trubey, P. (2019). Machine learning and sampling scheme: An empirical study of money laundering detection. *Computational Economics, 54*(3), 1043–1063.